OS Orange Problem – Unit 4

Name: Taran S

SRN: PES2UG24CS555

Section: I

Screenshot 1:

```
[04/20/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ make test_objects
gcc -o test_objects test_objects.o object.o -lcrypto
[04/20/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 2:

```
[04/20/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Screenshot 3:

```
[04/20/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 4:

```
[04/21/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a        blob 6.hello.
```

Screenshot 5:

```
[04/21/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gdb_history
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  peda-session-pes.txt
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
```

Screenshot 6:

```
[04/21/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ cat .pes/index
100755 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776797056 6 file1.txt
100755 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776797056 6 file2.txt
```

Screenshot 7:

```
[04/21/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ ./pes log
commit 32959d68ae9aea94aa90e0589887ee6f44484ff3949c785dc72cc87e71b5998d
Author: Taran S PES2UG24CS555
Date:   1776797823

    Add farewell

commit 88983203136a44b1f09235247aae4fd10e6dc3c12b377808ad815b658b3364c7
Author: Taran S PES2UG24CS555
Date:   1776797823

    Add world

commit c6323f93384260f84accce41b4db9ca5b4a1cada3ea5f079e2d51f7740a8d9c5
Author: Taran S PES2UG24CS555
Date:   1776797823

    Initial commit
```

Screenshot 8:

```
[04/21/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/32/959d68ae9aea94aa90e0589887ee6f44484ff3949c785dc72cc87e71b5998d
.pes/objects/48/1407f7237f7bd749dca31e2d8422bafe211741d5556f007e0faf821ffc7179
.pes/objects/59/bbe96a2ed004036dced1ff6a7930cf3c6e1c2e8aa95e71351db0d4dc235807
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7f/4cf3069795134e6b549b984dc928abf2616544f18714bf15d1fda61b75a090
.pes/objects/88/983203136a44b1f09235247aae4fd10e6dc3c12b377808ad815b658b3364c7
.pes/objects/c6/323f93384260f84accce41b4db9ca5b4a1cada3ea5f079e2d51f7740a8d9c5
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e5/99016fea35a210e69bf0bc22462820b225fad50789b89869042cd637293404
.pes/refs/heads/main
```

Screenshot 9:

```
[04/21/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ cat .pes/refs/heads/main
32959d68ae9aea94aa90e0589887ee6f44484ff3949c785dc72cc87e71b5998d
[04/21/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ cat .pes/HEAD
ref: refs/heads/main
```

Screenshot 10:

```
[04/21/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ sed -i 's/\r//' test_sequence.sh
[04/21/26]seed@VM:.../sf_PES2UG24CS555-pes-vc$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 47328352564f... Initial commit

Log after first commit:
commit 47328352564f23e7ad864beab7d67781a82fec82f742f09941ee8a0ed331088a
Author: Taran S PES2UG24CS555
Date:   1776828776

    Initial commit


--- Second Commit ---
Committed: 6508f20be348... Update file.txt

--- Third Commit ---
Committed: 41400011aaab... Add farewell

--- Full History ---
commit 41400011aaab795c1c3d82fc41eb199de7c10da6c1bb721095e9acea4e86a9ea
Author: Taran S PES2UG24CS555
Date:   1776828776

    Add farewell

commit 6508f20be3487620d0e921817b5e2a46f7371844f9200c0056e7cb4d42f5a7d7
Author: Taran S PES2UG24CS555
Date:   1776828776

    Update file.txt

commit 47328352564f23e7ad864beab7d67781a82fec82f742f09941ee8a0ed331088a
Author: Taran S PES2UG24CS555
Date:   1776828776

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
41400011aaab795c1c3d82fc41eb199de7c10da6c1bb721095e9acea4e86a9ea

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/41/400011aaab795c1c3d82fc41eb199de7c10da6c1bb721095e9acea4e86a9ea
.pes/objects/47/328352564f23e7ad864beab7d67781a82fec82f742f09941ee8a0ed331088a
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/65/08f20be3487620d0e921817b5e2a46f7371844f9200c0056e7cb4d42f5a7d7
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e88f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d8/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Q5.1: Implementing pes checkout <branch>

To implement pes checkout <branch>, the following steps are needed:

First, read the target branch file at .pes/refs/heads/<branch> to get the commit hash. Then read that commit object to get its tree hash. Next, update .pes/HEAD to contain ref: refs/heads/<branch>. Finally, reconstruct the working directory by walking the tree object and writing each file's blob contents to disk.

The files that change in .pes/ are: HEAD (updated to point to the new branch) and potentially the index (updated to match the new branch's tree).

What makes this operation complex is that the working directory must be updated to match the target branch's snapshot. This means files that exist in the current branch but not in the target must be deleted, files that exist in the target but not in the current branch must be created, and files that differ between branches must be overwritten. This requires a full diff between the current tree and the target tree, which involves reading and comparing multiple tree and blob objects recursively.

Q5.2: Detecting Dirty Working Directory Conflicts

When switching branches, a conflict occurs if a file that differs between the two branches has also been modified in the working directory since it was last staged.

To detect this using only the index and object store:

1. For each file in the current index, compare the file's current mtime and size on disk against the values stored in the index entry. If they differ, the file has been modified in the working directory and is considered dirty.
2. Then check if that same file's blob hash differs between the current branch's tree and the target branch's tree. If both conditions are true — the file is dirty locally AND it differs between branches — checkout must refuse to proceed to avoid overwriting the user's changes.

This approach uses the index as a fast-change-detection cache (using mtime and size) without needing to re-hash every file on disk.

Q5.3: Detached HEAD

Detached HEAD means that .pes/HEAD contains a commit hash directly instead of a branch reference like ref: refs/heads/main. This happens when you checkout a specific commit rather than a branch.

If you make commits in this state, the new commits are created and chained correctly, but no branch pointer is updated to track them. The commits exist in the object store but are only reachable through the hash stored in HEAD. If you then checkout a branch, HEAD is updated to point to that branch and the detached commits become unreachable — they have no branch pointing to them.

To recover those commits, you need the hash of the last commit you made while in detached HEAD state. You can then create a new branch pointing to it: pes branch <new-branch> <hash>. This makes the commits reachable again by giving them a named branch reference.

Q6.1: Garbage Collection Algorithm

To find and delete unreachable objects, the algorithm works as follows:

Start by collecting all reachable objects using a graph traversal. Beginning from every branch reference in .pes/refs/heads/, read each commit object. For each commit, mark its tree as reachable, then recursively mark all blobs and subtrees that the tree points to. Also follow each commit's parent pointer and repeat until there are no more parents. Use a hash set (such as a C hash table or a sorted array of ObjectIDs) to track all reachable hashes efficiently, since hash lookup is O(1).

Once all reachable objects are marked, scan every file under .pes/objects/ using find. Any file whose name (reconstructed as a full 64-char hex hash) is not in the reachable set is unreachable and can be safely deleted.

For a repository with 100,000 commits and 50 branches: starting from 50 branch tips, the traversal visits all 100,000 commits. Each commit points to a tree, and each tree may point to several blobs and subtrees. Assuming an average of 10 objects per commit (1 commit + 1 root tree + ~8 blobs/subtrees), you would visit approximately 1,000,000 objects in total.

Q6.2: Race Condition Between GC and Concurrent Commit

Consider this sequence of events:

1. A commit operation begins. It calls tree_from_index(), which writes several new blob and tree objects to the object store. At this point, these objects exist on disk but no commit object references them yet, and HEAD has not been updated.
2. GC runs concurrently. It scans all branch references and marks reachable objects. Since HEAD has not been updated yet, the new blob and tree objects written in step 1 are not reachable from any branch. GC marks them as unreachable and deletes them.
3. The commit operation continues. It tries to write the commit object referencing the tree hash from step 1, then calls head_update(). But the tree object has already been deleted by GC. The repository is now corrupt.

Git avoids this race condition by using a grace period during garbage collection. Objects that were created recently (within the last 2 weeks by default) are never deleted by GC, even if they appear unreachable. This gives any in-progress operations enough time to complete and update the branch references before GC can remove the objects they depend on.